Firewalll ruleset configuration

🛡️ Linux Firewall & iptables — Complete Lab Notes

> **Tags:** #networking #linux #firewall #iptables #ubuntu #fedora #lab
> **OS Coverage:** Ubuntu / Debian · Fedora / RHEL
> **Level:** Beginner → Intermediate

📖 Table of Contents

1. What is a Firewall?
2. What is iptables?
3. The Architecture — Tables, Chains & Rules
4. Understanding the Command Structure
5. Breaking Down Every Command from the Lab
6. Common Use Cases & Practical Scenarios
7. Ubuntu vs Fedora — Side-by-Side Reference
8. Persistence — Saving Rules Across Reboots
9. Quick Reference Cheat Sheet
10. Common Pitfalls & Gotchas

🔥 What is a Firewall?

A **firewall** is a security mechanism that monitors and controls incoming and outgoing **network traffic** based on a set of rules. Think of it as a bouncer at the door of your system — it decides what packets are allowed in, what goes out, and what gets silently dropped.

In Linux, the kernel's built-in packet filtering system is called **Netfilter**, and `iptables` is the command-line tool used to configure Netfilter rules.

⚙️ What is iptables?

`iptables` is a **user-space utility** that lets you configure the Linux kernel firewall. It works by inspecting each network packet that enters, passes through, or leaves your machine and applying rules to them.

```
Network Packet → Netfilter (kernel) → iptables rules → ACCEPT / DROP / REJECT
```

> **Note for Fedora/RHEL users:** Modern Fedora uses **nftables** under the hood, with `firewalld` as the high-level manager. However, `iptables` commands still work via a compatibility layer. Both are covered in this note.

🏗️ The Architecture — Tables, Chains & Rules

Tables

iptables organizes rules into **tables**, each serving a different purpose:

Table	Purpose
`filter`	Default table — decides whether to ACCEPT or DROP packets
`nat`	Network Address Translation — modifying source/destination addresses
`mangle`	Altering packet headers (TTL, TOS, etc.)
`raw`	Exempting packets from connection tracking

> **In your lab, you were working with the `filter` table (the default).**

Chains

Each table contains **chains** — ordered lists of rules. The default `filter` table has 3 chains:

```
┌──────────────────────────────────────────────────────────┐
│                      FILTER TABLE                        │
│                                                          │
│  ┌──────────┐     ┌──────────┐     ┌──────────────────┐  │
│  │  INPUT   │     │ FORWARD  │     │     OUTPUT       │  │
│  │          │     │          │     │                  │  │
│  │ Packets  │     │ Packets  │     │ Packets leaving  │  │
│  │ coming   │     │ routed   │     │ from this        │  │
│  │ INTO     │     │ THROUGH  │     │ machine          │  │
│  │ this box │     │ this box │     │                  │  │
│  └──────────┘     └──────────┘     └──────────────────┘  │
└──────────────────────────────────────────────────────────┘
```

- **INPUT** — Packets **destined for this machine** (e.g., someone connecting to your SSH server)

- **FORWARD** — Packets **passing through** this machine (used when acting as a router)
- **OUTPUT** — Packets **generated by this machine** going out

Rules & Targets

Each rule in a chain has:

- A **match** — criteria the packet must meet (source IP, destination IP, protocol, interface, etc.)
- A **target** — what to do if matched:

Target	Meaning
`ACCEPT`	Allow the packet through
`DROP`	Silently discard the packet (sender gets no response)
`REJECT`	Discard and send an error back to sender
`LOG`	Log the packet to syslog
`RETURN`	Stop processing this chain, return to caller

Policy (Default Action)

Each chain has a **default policy** — what happens to packets that match **no rules**. Shown in the `(policy ACCEPT)` part of `iptables -nvxL`.

🔬 Understanding the Command Structure

General Syntax

```
sudo iptables [-t table] COMMAND [chain] [match criteria] [-j target]
```

Part	Meaning
`-t table`	Which table (omit for `filter`)
`COMMAND`	What to do (-A, -D, -L, -I, -R, -F, -P...)
`chain`	Which chain (INPUT, OUTPUT, FORWARD)
`match criteria`	What packets to match (flags below)
`-j target`	Jump to this target if matched

Key Command Flags

Flag	Long Form	Meaning
`-A`	`--append`	Add rule to END of chain
`-I`	`--insert`	Insert rule at position (default: top)
`-D`	`--delete`	Delete a rule
`-L`	`--list`	List rules
`-F`	`--flush`	Delete ALL rules in chain
`-P`	`--policy`	Set default chain policy
`-n`	`--numeric`	Show IPs/ports as numbers (no DNS lookup)
`-v`	`--verbose`	Show packet/byte counters and interface info
`-x`	`--exact`	Show exact (unrounded) byte counts
`-d`	`--destination`	Match destination IP address
`-s`	`--source`	Match source IP address
`-p`	`--protocol`	Match protocol (tcp, udp, icmp)
`-i`	`--in-interface`	Match incoming network interface
`-o`	`--out-interface`	Match outgoing network interface
`-j`	`--jump`	Specify target

🧪 Breaking Down Every Command from the Lab

1. `sudo iptables -nvxL`

Purpose: View ALL current firewall rules with detailed statistics.

```
sudo iptables -nvxL
```

Breaking it down:

Flag	What it does
`sudo`	Run as root (required — iptables needs root)
`-n`	**Numeric** output — don't resolve IPs to hostnames (faster, clearer)
`-v`	**Verbose** — shows packet counters, byte counters, and interfaces
`-x`	**Exact** — shows full byte count numbers without rounding (e.g., `1048576` instead of `1.0M`)
`-L`	**List** all chains and rules

Sample Output Explained:

```
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination
```

- `Chain INPUT` → This is the INPUT chain
- `(policy ACCEPT 0 packets, 0 bytes)` → Default action is ACCEPT; 0 packets have been processed (no rules set yet)
- `pkts` → Number of packets matched by this rule
- `bytes` → Total bytes of matched packets
- `target` → What happens to matched packets
- `prot` → Protocol (tcp/udp/icmp/all)
- `opt` → Options
- `in` / `out` → Interfaces (e.g., `eth0`, `lo`)
- `source` / `destination` → IP addresses

When to use it: Always run this first to inspect current firewall state before making changes.

---

2. `sudo iptables -nvxL INPUT`

Purpose: Same as above, but **only shows the INPUT chain**.

```
sudo iptables -nvxL INPUT
```

Useful when you only care about inbound traffic rules. Equivalent commands for other chains:

```
sudo iptables -nvxL OUTPUT
sudo iptables -nvxL FORWARD
```

---

3. `sudo iptables -A INPUT`

Purpose: Append a rule to the INPUT chain (incomplete — shown for educational purposes).

```
sudo iptables -A INPUT
```

> ⚠️ **This command as-is is INVALID** — `-A INPUT` requires match criteria and a target. Running it alone throws an error. It was likely shown to demonstrate the syntax pattern.

A complete example would be:

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
# Meaning: Append to INPUT a rule that ACCEPTs TCP packets on port 22 (SSH)
```

---

4. `sudo iptables -D INPUT`

Purpose: Delete a rule from the INPUT chain.

```
sudo iptables -D INPUT
```

> ⚠️ Also incomplete as shown. There are two ways to properly delete rules:

Method 1 — Delete by rule specification:

```
sudo iptables -D INPUT -d 127.1.2.3 -j DROP
# Deletes the rule that matches exactly this specification
```

Method 2 — Delete by rule number:

```
sudo iptables -nvxL INPUT --line-numbers   # First, see rule numbers
sudo iptables -D INPUT 1                   # Then delete rule #1
```

---

5. `sudo iptables -A INPUT -d 127.1.2.3 -j DROP`

Purpose: Drop ALL packets destined for the IP address `127.1.2.3`.

```
sudo iptables -A INPUT -d 127.1.2.3 -j DROP
```

Breaking it down:

Part	Meaning
`-A INPUT`	Append this rule to the INPUT chain
`-d 127.1.2.3`	Match packets with **destination** IP = `127.1.2.3`
`-j DROP`	**Drop** matching packets (silently discard)

What this achieves:
Any packet trying to reach `127.1.2.3` on this machine will be silently discarded. The sender gets no response — the packet just disappears.

Why `127.1.2.3`?
The `127.x.x.x` range is the **loopback** range. `127.0.0.1` is the standard loopback, but the entire `127.0.0.0/8` subnet routes to localhost. This rule demonstrates that even within the loopback range, you can selectively block specific addresses.

Proof it works (from your lab):

```
ping 127.0.0.1    # Works — not blocked
ping 127.1.2.3    # Blocked — packets dropped, no response
```

---

6. `ping -I 172.17.100.16 172.17.100.100`

Purpose: Send ping from a **specific source interface/IP** to a destination.

```
ping -I 172.17.100.16 172.17.100.100
```

Part	Meaning
`ping`	Send ICMP echo requests
`-I 172.17.100.16`	Use this IP as the **source interface** (bind to this local IP)
`172.17.100.100`	The **destination** to ping

Why this matters:
If your machine has multiple network interfaces (e.g., `eth0` = `172.17.100.16` and `wlan0` = `10.34.0.163`), you can control which interface sends the ping. This is useful for:

- Testing connectivity through a specific network path
- Diagnosing routing issues
- Testing firewall rules on specific interfaces

Variations from your lab:

```
ping -I 172.17.100.1 172.17.100.100   # Use gateway IP as source
ping -I 10.34.0.163 172.17.100.100    # Use WiFi IP as source
ping -4 172.17.100.100                 # Force IPv4 (not IPv6)
```

---

7. `sudo iptables -A INPUT -i lo -j DROP`

Purpose: Drop ALL packets coming in on the loopback interface (`lo`).

```
sudo iptables -A INPUT -i lo -j DROP
```

Part	Meaning
`-A INPUT`	Append to INPUT chain
`-i lo`	Match packets **incoming on interface `lo`** (loopback)
`-j DROP`	Drop them

What `lo` is:
`lo` is the **loopback interface** — a virtual network interface that loops back to the same machine. It has the IP `127.0.0.1`. Many local services communicate via `lo` (databases, local web servers, etc.).

> ⚠️ **WARNING: This is a dangerous rule in production!**
> Dropping all loopback traffic will break many local services. For example:
>
> - MySQL/PostgreSQL connections via `localhost` will fail
> - Local web apps communicating internally will break
> - Some system tools rely on loopback communication

Why it's useful in a lab:
It demonstrates how interface-based rules work, and shows the difference between IP-based blocking (`-d 127.1.2.3`) vs. interface-based blocking (`-i lo`).

🎯 Common Use Cases & Practical Scenarios

Use Case 1: Allow SSH, Block Everything Else

```
# Allow established connections first (CRITICAL — always do this first!)
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allow SSH from anywhere
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow loopback
sudo iptables -A INPUT -i lo -j ACCEPT

# Drop everything else
sudo iptables -P INPUT DROP
```

> 🔑 **Key Rule:** Always add the `ESTABLISHED,RELATED` rule FIRST, or you'll cut off your own SSH session!

Use Case 2: Block a Specific IP

```
# Block all traffic from IP 192.168.1.50
sudo iptables -A INPUT -s 192.168.1.50 -j DROP

# Block traffic to a specific destination
sudo iptables -A INPUT -d 10.0.0.5 -j DROP
```

Use Case 3: Block a Port

```
# Block incoming HTTP traffic (port 80)
sudo iptables -A INPUT -p tcp --dport 80 -j DROP

# Block incoming UDP traffic on port 53 (DNS)
sudo iptables -A INPUT -p udp --dport 53 -j DROP
```

Use Case 4: Allow a Subnet, Block Others

```
# Allow traffic from 192.168.1.0/24 subnet
sudo iptables -A INPUT -s 192.168.1.0/24 -j ACCEPT

# Block everything else
sudo iptables -P INPUT DROP
```

Use Case 5: Log and Drop Suspicious Traffic

```
# Log the packet first (writes to /var/log/syslog or kern.log)
sudo iptables -A INPUT -s 10.0.0.99 -j LOG --log-prefix "SUSPICIOUS: " --log-
level 4

# Then drop it
sudo iptables -A INPUT -s 10.0.0.99 -j DROP
```

Use Case 6: Rate Limiting (Anti-DDoS / Port Scan Protection)

```
# Limit new SSH connections to 3 per minute
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
  -m limit --limit 3/min --limit-burst 3 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -m state --state NEW -j DROP
```

Use Case 7: Flush (Reset) All Rules

```
# Delete all rules in all chains
sudo iptables -F

# Also reset policies to ACCEPT (important to avoid lockout!)
sudo iptables -P INPUT ACCEPT
sudo iptables -P FORWARD ACCEPT
sudo iptables -P OUTPUT ACCEPT
```

🐧 Ubuntu vs Fedora — Side-by-Side Reference

Installation

Task	Ubuntu / Debian	Fedora / RHEL
iptables available by default	✅ Yes	✅ Yes (via compat layer)
Preferred firewall tool	`ufw` (wraps iptables)	`firewalld` (wraps nftables)
Install iptables	`sudo apt install iptables`	`sudo dnf install iptables`
Install iptables-persistent	`sudo apt install iptables-persistent`	`sudo dnf install iptables-services`

Viewing Rules

Task	Ubuntu	Fedora
List all rules	`sudo iptables -nvxL`	`sudo iptables -nvxL`

Task	Ubuntu	Fedora
List with line numbers	`sudo iptables -nvxL --line-numbers`	`sudo iptables -nvxL --line-numbers`
View IPv6 rules	`sudo ip6tables -nvxL`	`sudo ip6tables -nvxL`
View using nftables	`sudo nft list ruleset`	`sudo nft list ruleset`
View firewalld rules	N/A (not default)	`sudo firewall-cmd --list-all`

Managing Rules

Task	Ubuntu	Fedora
Append rule	`sudo iptables -A INPUT ...`	`sudo iptables -A INPUT ...`
Insert at top	`sudo iptables -I INPUT 1 ...`	`sudo iptables -I INPUT 1 ...`
Delete rule	`sudo iptables -D INPUT ...`	`sudo iptables -D INPUT ...`
Flush all rules	`sudo iptables -F`	`sudo iptables -F`
Set default policy	`sudo iptables -P INPUT DROP`	`sudo iptables -P INPUT DROP`

> iptables syntax is **identical** on Ubuntu and Fedora. The differences are in **saving/loading** rules and the **default firewall manager**.

Saving & Persisting Rules

Ubuntu / Debian

```
# Method 1: Using iptables-persistent (recommended)
sudo apt install iptables-persistent

# Save current rules
sudo netfilter-persistent save
# Rules saved to:
# /etc/iptables/rules.v4  (IPv4)
# /etc/iptables/rules.v6  (IPv6)

# Reload saved rules
sudo netfilter-persistent reload

# Method 2: Manual save/restore
```

```
sudo iptables-save > /etc/iptables/rules.v4
sudo iptables-restore < /etc/iptables/rules.v4
```

Fedora / RHEL / CentOS

```
# Method 1: Using iptables-services
sudo dnf install iptables-services
sudo systemctl enable iptables    # Enable on boot
sudo systemctl start iptables

# Save current rules
sudo service iptables save
# Rules saved to: /etc/sysconfig/iptables

# Start/Stop/Restart
sudo systemctl start iptables
sudo systemctl stop iptables
sudo systemctl restart iptables

# Method 2: Using iptables-save
sudo iptables-save > /etc/sysconfig/iptables
```

Using firewalld (Fedora's Native Tool)

Fedora prefers `firewalld` — a higher-level firewall manager. Here are the equivalents:

```
# Check firewalld status
sudo systemctl status firewalld

# Start/enable firewalld
sudo systemctl start firewalld
sudo systemctl enable firewalld

# List all active rules
sudo firewall-cmd --list-all

# Add a service (e.g., SSH)
sudo firewall-cmd --permanent --add-service=ssh

# Add a specific port
sudo firewall-cmd --permanent --add-port=8080/tcp

# Remove a port
sudo firewall-cmd --permanent --remove-port=8080/tcp
```

```
# Block a source IP
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source
address="192.168.1.50" drop'

# Reload to apply permanent changes
sudo firewall-cmd --reload

# List all zones
sudo firewall-cmd --get-zones

# Set default zone
sudo firewall-cmd --set-default-zone=public
```

> **Note:** `firewalld` and `iptables` can conflict. Disable one before using the other.
>
> ```
> sudo systemctl stop firewalld
> sudo systemctl disable firewalld
> ```

Using ufw (Ubuntu's Native Tool)

Ubuntu prefers `ufw` (Uncomplicated Firewall) — a simpler interface over iptables:

```
# Check status
sudo ufw status verbose

# Enable/disable
sudo ufw enable
sudo ufw disable

# Allow SSH
sudo ufw allow ssh        # or: sudo ufw allow 22/tcp

# Allow a port
sudo ufw allow 8080/tcp

# Deny a port
sudo ufw deny 23/tcp

# Block an IP
sudo ufw deny from 192.168.1.50

# Allow from subnet
sudo ufw allow from 192.168.1.0/24

# Delete a rule
```

```
sudo ufw delete allow 8080/tcp

# Reset all rules
sudo ufw reset
```

💾 Persistence — Saving Rules Across Reboots

> ⚠️ **IMPORTANT:** iptables rules are lost on reboot unless explicitly saved!

Ubuntu — Quick Setup

```
sudo apt install iptables-persistent -y
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT   # example rule
sudo netfilter-persistent save
```

Fedora — Quick Setup

```
sudo dnf install iptables-services -y
sudo systemctl enable --now iptables
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT   # example rule
sudo service iptables save
```

📋 Quick Reference Cheat Sheet

```
# ─── VIEWING ──────────────────────────────────────
sudo iptables -nvxL                    # View all rules (verbose + exact)
sudo iptables -nvxL INPUT              # View INPUT chain only
sudo iptables -nvxL --line-numbers     # View with rule numbers
sudo iptables -t nat -nvxL             # View NAT table

# ─── ADDING RULES ─────────────────────────────────
sudo iptables -A INPUT -s IP -j DROP           # Block source IP
sudo iptables -A INPUT -d IP -j DROP           # Block destination IP
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT   # Allow SSH
sudo iptables -A INPUT -i lo -j ACCEPT         # Allow loopback
sudo iptables -A INPUT -i eth0 -j DROP         # Block all on eth0
sudo iptables -A OUTPUT -d IP -j DROP          # Block outbound to IP

# ─── DELETING RULES ───────────────────────────────
sudo iptables -D INPUT -s IP -j DROP           # Delete by spec
sudo iptables -D INPUT 3                       # Delete rule #3
```

```
sudo iptables -F INPUT                          # Flush INPUT chain
sudo iptables -F                                # Flush ALL rules

# ─── POLICIES ──────────────────────────────────────────
sudo iptables -P INPUT DROP            # Default: drop incoming
sudo iptables -P INPUT ACCEPT          # Default: accept incoming
sudo iptables -P OUTPUT DROP           # Default: drop outgoing

# ─── SAVING ────────────────────────────────────────────
# Ubuntu:
sudo netfilter-persistent save
# Fedora:
sudo service iptables save

# ─── PING UTILITIES ────────────────────────────────────
ping -4 IP                             # Force IPv4 ping
ping -I INTERFACE_IP DEST_IP           # Ping from specific interface
ping -c 4 IP                           # Send exactly 4 pings
```

⚠️ Common Pitfalls & Gotchas

1. Locking Yourself Out of SSH

```
# ❌ DANGER — This locks you out immediately!
sudo iptables -P INPUT DROP   # Without first allowing SSH!

# ✅ SAFE — Allow SSH before setting default DROP
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
sudo iptables -P INPUT DROP   # NOW safe to set DROP policy
```

2. Rule Order Matters

Rules are evaluated **top to bottom** — first match wins.

```
# ❌ This NEVER drops 192.168.1.50, because the ACCEPT above matches first
sudo iptables -A INPUT -s 192.168.1.0/24 -j ACCEPT
sudo iptables -A INPUT -s 192.168.1.50 -j DROP   # Never reached!

# ✅ Put specific rules BEFORE general ones
sudo iptables -A INPUT -s 192.168.1.50 -j DROP   # Specific first
sudo iptables -A INPUT -s 192.168.1.0/24 -j ACCEPT
```

3. Rules Are NOT Persistent by Default

After reboot, all rules vanish unless saved with `iptables-persistent` or `iptables-services`.

4. DROP vs REJECT

```
-j DROP     # Silent — sender waits for timeout (no error message)
-j REJECT   # Polite — sends "connection refused" back to sender
```

Use `DROP` for security (don't reveal you exist). Use `REJECT` for debugging/usability.

5. Flushing Rules with Default Policy DROP

```
# ❌ DANGER — If policy is DROP and you flush, you're locked out!
sudo iptables -P INPUT DROP
sudo iptables -F    # Now nothing gets in, including SSH!

# ✅ Always reset policy to ACCEPT BEFORE flushing
sudo iptables -P INPUT ACCEPT
sudo iptables -F
```

6. IPv4 vs IPv6

`iptables` only handles IPv4. For IPv6, use `ip6tables`:

```
sudo ip6tables -nvxL
sudo ip6tables -A INPUT -p tcp --dport 22 -j ACCEPT
```

🔗 Related Topics to Explore

- `nftables` — the modern replacement for iptables (default on Fedora 32+)
- `firewalld` — zone-based firewall management (Fedora/RHEL)
- `ufw` — simplified firewall for Ubuntu
- `tcpdump` — packet capture to verify firewall behavior
- `netstat` / `ss` — view active connections
- Network namespaces — isolated network stacks in Linux
- `conntrack` — connection tracking module for stateful firewalls

Notes compiled from networking lab exercises — Ubuntu 24.x / Fedora 40+